

HELIOS-NODE

Controlador predictivo de microrred DC solar

Antonio Zambudio Rodriguez · drizzlyrdrgz.exe@protonmail.com
github.com/QuantumDrizzy/HELIOS · Licencia MIT

¿Qué es HELIOS?

HELIOS es un controlador predictivo de microrred DC solar construido completamente desde cero. Combina un controlador MPPT en tiempo real escrito en Rust con un agente de inteligencia artificial (CNN-LSTM) que predice la irradiancia solar usando datos reales de PVGIS — la base de datos climática oficial de la Comisión Europea.

El sistema funciona en simulación completa sin necesidad de hardware físico, pero está diseñado para conectarse a sensores reales (INA219 vía I2C, GPIO en Raspberry Pi o STM32) con mínimas modificaciones.

Arquitectura

Dos procesos independientes se comunican a través de una base de datos SQLite en modo WAL (Write-Ahead Logging), que actúa como bus IPC de baja latencia:

Proceso 1 — Rust Core (tick 100 ms)

Ejecuta el algoritmo MPPT Perturb & Observe con bias predictivo. Simula la física del panel (V·I, potencia, duty cycle PWM). Lee el forecast del agente AI desde SQLite cada ciclo y escribe telemetría completa (V, I, P, duty, SOC, irradiancia) en la base de datos. Mantiene un audit log SHA-256 encadenado con integridad tamper-evident. Las migraciones de esquema se aplican automáticamente al arrancar.

Proceso 2 — AI Agent Python (tick 1 s)

Lee los últimos N registros de telemetría de SQLite. Normaliza la irradiancia real (irradiance_wm2 / 1100) para inferencia LSTM. Escribe el forecast normalizado [0,1] en la tabla ai_forecasts. El Rust Core lo lee y ajusta el step del P&O en consecuencia.

Diagrama de flujo

```
PVGIS Data (Murcia, 8760h) → dataset_generator.py → train.py → helios_predictor.pt  
→ agent.py (--serve) → energy_bus.sqlite ← helios-core (Rust) → dashboard equi
```

Modelo LSTM

El forecaster combina una capa CNN de extracción de características con capas LSTM para modelado temporal. El input son secuencias de irradiancia normalizada (GHI/1100) y el output es el forecast del siguiente periodo, normalizado en [0,1].

Entrenado con el TMY (Typical Meteorological Year) de PVGIS para Aljucer, Murcia — 8760 horas de irradiancia solar real. El generador de dataset añade perturbaciones de nube y ruido de sensor para dotar al modelo de robustez ante condiciones reales. Tiempo de entrenamiento: ~5 minutos en CPU.

Datos Reales: PVGIS

PVGIS (Photovoltaic Geographical Information System) es la herramienta oficial de la Comisión Europea para datos solares. HELIOS descarga el TMY para Murcia y lo usa para simular ciclos día/noche realistas, entrenar el LSTM y alimentar el controlador Rust con irradiancia por hora. La simulación no es aleatoria — reproduce el comportamiento solar real de Murcia a lo largo del año. Cambiar la ubicación es un único parámetro en `ai/pvgis_client.py`.

MPPT con Bias Predictivo

El P&O clásico es reactivo: perturba el duty cycle y observa si la potencia sube o baja. HELIOS lo mejora con un bias predictivo:

$$\text{duty_step} = \text{base_step} \times (1 + \alpha \times \text{forecast})$$

Cuando el forecast predice alta irradiancia, el step se amplía para converger antes al MPP. Cuando predice una caída (nube), el step se reduce para evitar oscilaciones. El beneficio real aparece en transiciones climáticas, donde el P&O clásico pierde potencia por perturbaciones en el momento equivocado.

Base de Datos SQLite — Esquema

5 tablas:

Tabla	Contenido	Frecuencia
power_telemetry	V, I, P, duty, SOC, irradiance_wm2	100 ms
ai_forecasts	forecast [0,1] + inference_time_ms	1 s
material_states	resultados simulación cuántica (SUBSTRATE)	bajo demanda
system_events	alertas, cambios de configuración	eventos
audit_log	log SHA-256 encadenado (tamper-evident)	cada escritura

Stack Tecnológico

Capa	Tecnología
Controlador MPPT	Rust + tokio (async runtime)
Dashboard	egui (immediate mode GUI)
Base de datos / IPC	SQLite WAL + sqlx
Audit log	SHA-256 encadenado (sha2 crate)
Post-quantum (investigación)	ML-KEM 768 + ML-DSA (NIST 2024)

AI forecaster	Python + PyTorch CNN-LSTM
Datos solares	PVGIS API (Comisión Europea)
Deploy hardware	Cross-compile Rust → Raspberry Pi

Seguridad Post-Cuántica: helios-sentinel

helios-sentinel es el daemon de seguridad de HELIOS. Implementa los algoritmos criptográficos post-cuánticos estandarizados por el NIST en 2024, diseñados para resistir ataques de computadores cuánticos — incluso ante un adversario con acceso a un ordenador cuántico de gran escala.

Algoritmos implementados

Algoritmo	Estándar NIST	Función en HELIOS
ML-KEM 768	FIPS 203 (2024)	Intercambio de claves post-cuántico entre nodos
ML-DSA	FIPS 204 (2024)	Firma digital de checkpoints del modelo AI

¿Por qué es relevante?

Los sistemas de gestión energética almacenan telemetría crítica durante años. Un atacante con acceso futuro a computación cuántica podría romper la criptografía clásica (RSA, ECDSA) y falsificar datos históricos retroactivamente. ML-KEM y ML-DSA son resistentes a este vector de ataque por diseño matemático — están basados en problemas de retículos (lattices) para los que no existe algoritmo cuántico eficiente conocido.

Función específica en HELIOS

helios-sentinel firma los checkpoints del modelo LSTM con ML-DSA antes de guardarlos en disco. Al arrancar el sistema, verifica la firma antes de cargar el modelo. Si alguien modificó el archivo del modelo entre sesiones — intencionalmente o por corrupción — el sistema lo detecta y rechaza el checkpoint. En instalaciones reales donde la telemetría tiene valor legal o regulatorio, la firma post-cuántica garantiza la cadena de custodia de los datos.

Estado actual

helios-sentinel está completamente implementado con criptografía real. La firma ML-DSA-65 produce 3309 bytes de firma real sobre un hash SHA3-256 del checkpoint. El seed de 32 bytes se persiste en disco — la clave es estable entre reinicios. El binario helios-verify permite verificar cualquier checkpoint firmado de forma independiente. Los tests de roundtrip (sign → verify) pasan en CI. El único componente que requiere Linux para operar es el Unix Domain Socket server — diseñado para producción en Raspberry Pi.

Pasos hacia Hardware Real

La arquitectura está diseñada para hardware desde el principio. El paso siguiente es cablear un sensor INA219 vía I2C y reemplazar las lecturas simuladas en rust/src/main.rs por lecturas reales del ADC. El repo incluye scripts de cross-compilation y despliegue SSH para Raspberry Pi listos para usar:

```
scripts/cross_compile_rpi.sh – compila el binario Rust para ARM
scripts/deploy_rpi.sh – despliega vía SSH a la RPi
```

Hardware mínimo para instalación real: Raspberry Pi 4, sensor INA219 (I2C), convertidor DC-DC con control PWM, panel solar + batería.

Estado Actual y Limitaciones

Lo que funciona: simulación completa día/noche con PVGIS real, forecaster LSTM activo, dashboard en tiempo real, audit log SHA-256, migraciones de DB automáticas.

Pendiente: lectura real de ADC/GPIO (INA219), firma ML-DSA real en helios-sentinel (actualmente mock), tests de integración y CI.